

PRÁCTICA 6

Servicio con almacenamiento local y remoto en .NET

RepositorioRemoto.Back

Lucía Fuertes Cruz
Laura Santamaria Parra

Entorno Servidor · 2º DAW
Curso 2026-2027

Índice

1. Introducción
2. Objetivos
3. Requisitos del proyecto
4. Estructura y tecnologías
5. Configuración e inyección de dependencias
6. Persistencia y caché
7. Funcionamiento del servicio
8. Sincronización, notificaciones y exportación
9. Validación, errores y logs
10. Pruebas y cobertura
11. Docker
12. Diagramas
13. Conclusiones

1. Introducción

Esta práctica consiste en desarrollar un servicio en .NET para gestionar usuarios utilizando una API REST remota, una base de datos local y una caché. La API utilizada es JSONPlaceholder y la aplicación mantiene una copia local de los datos para reducir llamadas innecesarias y poder trabajar con distintos entornos.

El proyecto se ha organizado por responsabilidades: servicio de usuarios, repositorio, caché, cliente Refit, validadores, exportación a JSON, notificaciones e infraestructura. El flujo principal que realiza operaciones de entrada/salida se ha implementado de forma asíncrona y los errores de negocio se devuelven mediante `Result<T, DomainError>`.

2. Objetivos

- Implementar el CRUD de usuarios de forma asíncrona.
- Seguir el orden de lectura caché → base de datos → API remota.
- Realizar las escrituras primero en la API remota y después en la base de datos local.
- Permitir dos configuraciones: Development con SQLite y MemoryCache, y Production con PostgreSQL y Redis.
- Sincronizar los datos locales con JSONPlaceholder cada 60 segundos.
- Añadir notificaciones reactivas en las operaciones de creación, actualización y borrado.
- Exportar los usuarios a un archivo JSON.
- Comprobar el comportamiento con tests y revisar la cobertura de código.

3. Requisitos del proyecto

3.1 Requisitos funcionales

ID	Requisito	Descripción
RF-01	Obtener usuarios	Consultar los usuarios de la base de datos local. Si no hay datos locales, obtenerlos desde JSONPlaceholder y devolverlos sin guardar la lista completa automáticamente.
RF-02	Obtener por ID	Buscar primero en caché, después en base de datos y por último en la API. Si se recupera de BD o API, guardarlo en caché.
RF-03	Crear usuario	Validar la petición, crear el usuario en la API, guardar el resultado en la BD local y emitir una notificación.

ID	Requisito	Descripción
RF-04	Actualizar usuario	Comprobar el ID, validar los datos, actualizar API y BD local y emitir una notificación.
RF-05	Eliminar usuario	Comprobar que existe, eliminarlo de API y BD, borrar su clave de caché y emitir una notificación.
RF-06	Sincronizar	Cada 60 segundos, limpiar caché y BD local y volver a cargar los usuarios de la API.
RF-07	Exportar JSON	Generar data/users.json con los usuarios obtenidos del repositorio local.

3.2 Requisitos no funcionales

ID	Requisito	Aplicación en el proyecto
RNF-01	Asincronía	El CRUD, la API, el repositorio, la caché, la exportación de archivos y la sincronización utilizan Task, async y await.
RNF-02	Mantenibilidad	El código se divide en interfaces, servicios, repositorios, validadores, mappers e infraestructura.
RNF-03	Configuración	El entorno se selecciona con appsettings.development.json o appsettings.production.json.
RNF-04	Testabilidad	Las dependencias se inyectan mediante interfaces y se pueden sustituir por mocks en los tests.
RNF-05	Despliegue	La aplicación puede ejecutarse con Docker y Docker Compose.
RNF-06	Trazabilidad	Serilog registra operaciones y errores en distintos niveles.

3.3 Requisitos de información

Elemento	Información almacenada
User	Id, Name, UserName, Email, Address, Phone, Website, Company y campos de auditoría.

Elemento	Información almacenada
Address	Street, Suite, City, ZipCode y Geo.
Geo	Lat y Lng.
Company	Name, CatchPhrase y Bs.
Notification	Tipo de operación, mensaje y fecha/hora.
Configuración	API, proveedor de base de datos, conexión, caché e intervalo de sincronización.
Exportación	Archivo users.json generado en la carpeta data.

4. Estructura y tecnologías

La solución separa las responsabilidades para evitar que UserService conozca detalles concretos de persistencia o infraestructura. Las clases principales son las siguientes:

Parte	Clases / tecnologías	Función
API remota	Refit · IApiJsonPlaceholder	Define las llamadas HTTP a JSONPlaceholder.
Servicio	UserService	Coordina el CRUD, la validación, caché, repositorio, API y notificaciones.
Persistencia	EF Core · UserEfcRepository	Acceso a los datos locales mediante un único repositorio.
Base de datos	SQLite / PostgreSQL	SQLite en Development y PostgreSQL en Production.
Caché	MemoryCache / Redis	Caché local en Development y distribuida en Production.
Notificaciones	System.Reactive	Publica eventos de create, update y delete.
Errores	CSharpFunctionalExtensions	Uso de Result<T, DomainError> para controlar errores esperados.
Testing	NUnit · Moq · FluentAssertions	Pruebas unitarias, mocks, Verify y aserciones legibles.

5. Configuración e inyección de dependencias

5.1 Entornos

La clase Configuracion carga el appsettings correspondiente al entorno indicado al arrancar. Si no se pasa ningún argumento se utiliza Development.

Configuración	Development	Production
Base de datos	SQLite	PostgreSQL
Caché	MemoryCache	Redis
API	JSONPlaceholder	JSONPlaceholder
Sincronización	60 s	60 s

5.2 Scrutor y ciclos de vida

Para no registrar manualmente todas las clases, se usa Scrutor con tres interfaces marcadoras: `ITransientService`, `IScopedService` e `ISingletonService`. Después, `DependenciesProviderExtension` registra de forma manual la base de datos, la caché y el cliente externo porque dependen del entorno.

Ciclo de vida	Uso en el proyecto
Transient	<code>RequestValidator</code> . No mantiene estado y puede crearse en cada resolución.
Scoped	<code>UserService</code> , <code>UserStorage</code> y <code>UserEfcRepository</code> . El repositorio comparte el ciclo de vida del <code>DbContext</code> .
Singleton	<code>ConsoleNotificationService</code> , <code>BackgroundService</code> y las implementaciones de caché.

6. Persistencia y caché

6.1 Entity Framework Core

El proyecto utiliza un único repositorio, `UserEfcRepository`, sobre `DbContext`. De esta forma no se duplica la lógica CRUD para SQLite y PostgreSQL. Lo que cambia es el contexto que se registra según el entorno.

- `AppDbContextSqlite` configura la tabla `users` y serializa `Address` y `Company` como JSON dentro de SQLite.
- `AppDbContextPostgre` usa la misma entidad, pero guarda `Address` y `Company` con tipo `jsonb`.
- Los dos contextos se configuran con Fluent API.
- Los mapeadores son manuales y convierten DTOs a modelos de dominio y objetos complejos a JSON cuando es necesario.

6.2 Caché

`ICache` define una interfaz común con `GetAsync`, `SetAsync`, `RemoveAsync` y `RemoveAllAsync`. La implementación concreta depende del entorno: `MemoryCache` en desarrollo y `Redis` en producción.

La caché se utiliza principalmente en las lecturas por ID. En Create y Update no se guarda automáticamente el resultado en caché: crear o modificar un usuario no implica que se vaya a consultar justo después. En Delete sí se elimina la clave para evitar devolver un dato antiguo.

7. Funcionamiento del servicio

UserService concentra la lógica de los casos de uso. Las operaciones principales de API, repositorio, caché, exportación y sincronización se realizan de forma asíncrona mediante Task, async y await. El flujo de lectura y escritura es el siguiente:

Operación	Flujo
GetAllAsync	BD local → si está vacía, API → devolver sin guardar toda la lista automáticamente.
GetByIdAsync	Caché → BD → API. Si se obtiene de BD o API, se guarda en caché.
CreateAsync	Mapear → validar → API → BD → notificación.
UpdateAsync	Comprobar ID → validar → comprobar existencia → API → BD → notificación.
DeleteAsync	Comprobar existencia → API → BD → borrar caché → notificación.
ExportToJsonAsync	BD → UserStorage → data/users.json.

Para Create y Update se usa Railway Oriented Programming con Map, Bind y Tap. El objetivo es encadenar pasos que pueden fallar sin llenar el servicio de comprobaciones y excepciones para errores esperados.

8. Sincronización, notificaciones y exportación

8.1 BackgroundService

BackgroundService utiliza PeriodicTimer con un intervalo de 60 segundos. En cada ciclo crea un scope, obtiene el repositorio y la API y realiza una sincronización completa.

- Limpia la caché.
- Vacía la tabla local de usuarios.
- Obtiene todos los usuarios de JSONPlaceholder.
- Inserta de nuevo los usuarios en la base de datos local.

8.2 Notificaciones con Rx.NET

ConsoleNotificationService contiene un Subject<Notification> y expone un IObservable<Notification>. Program.cs se suscribe al arrancar y UserService publica una notificación después de Create, Update y Delete.

8.3 Exportación a JSON

UserStorage se utiliza únicamente para la exportación. Recibe una colección de usuarios y, cuando se solicita la exportación, genera el archivo data/users.json con System.Text.Json. El archivo se escribe de forma asíncrona en UTF-8 y las propiedades se serializan en camelCase. El JSON no sustituye a la base de datos ni se utiliza como almacenamiento principal.

9. Validación, errores y logs

9.1 Validación

RequestValidator comprueba los datos generales del usuario y reutiliza AddressValidator y CompanyValidator. Primero se revisan valores nulos o vacíos y después el formato mediante expresiones regulares.

9.2 Errores

Todos los errores del proyecto parten de DomainError. Se han separado en errores de usuario, servicio, repositorio y exportación. Los métodos que pueden fallar devuelven Result<T, DomainError>, lo que permite tratar los errores esperados sin depender únicamente de try/catch.

9.3 Serilog

Serilog se utiliza para seguir la ejecución del programa. Se registran mensajes Debug para operaciones internas, Information para acciones relevantes, Warning para fallos recuperables de caché y Error para errores importantes como una sincronización o exportación fallida.

10. Pruebas y cobertura

Los tests están hechos con NUnit. Para aislar UserService y otros componentes se usa Moq, y las comprobaciones se escriben con FluentAssertions. Los tests siguen el patrón AAA (Arrange, Act, Assert) y en los mocks se utiliza Verify para comprobar que las dependencias se han llamado el número de veces esperado.

Zona probada	Ejemplos
DTOs y modelos	Constructores y asignación de propiedades.
Mappers	Conversión DTO→modelo, serialización JSON y casos inválidos.
Validadores	Campos vacíos, regex y datos correctos.
Caché	Get, Set, Remove y RemoveAll en MemoryCache y Redis.
DbContext y repositorio	Mapectos de EF Core y operaciones CRUD.
UserService	Flujos caché/BD/API, CRUD, exportación y notificaciones mediante mocks.

Zona probada	Ejemplos
BackgroundService	Cancelación y sincronización.
Exportación JSON	Generación y contenido del archivo JSON exportado.

10.1 Cobertura

Además de comprobar que los tests pasan, se revisa la cobertura con Report Coverage/ReportGenerator. Esto permite ver el porcentaje total y, sobre todo, qué clases, métodos y líneas concretas han sido ejecutadas por los tests. Así se pueden localizar partes del código que todavía no tienen una prueba asociada.

```
dotnet test /p:CollectCoverage=true /p:CoverletOutputFormat=cobertura
reportgenerator -reports:**/coverage.cobertura.xml -targetdir:coveragereport -reporttypes:Html
```

En Redis y PostgreSQL también se utilizan Testcontainers en varias pruebas para comprobar el comportamiento contra servicios reales dentro de contenedores temporales.

11. Docker

El proyecto incluye Dockerfile y docker-compose.yml. Docker Compose permite arrancar únicamente el backend en Development o añadir PostgreSQL y Redis mediante el perfil prod.

11.1 Development: SQLite + MemoryCache

Para ejecutar la configuración de desarrollo, el Dockerfile debe iniciar la aplicación con el argumento Development:

```
ENTRYPOINT ["dotnet", "RepositorioRemoto.Back.dll", "Development"]

docker compose up -d --build
```

11.2 Production: PostgreSQL + Redis

Para producción se cambia el argumento del ENTRYPOINT a Production y se levanta el perfil prod:

```
ENTRYPOINT ["dotnet", "RepositorioRemoto.Back.dll", "Production"]

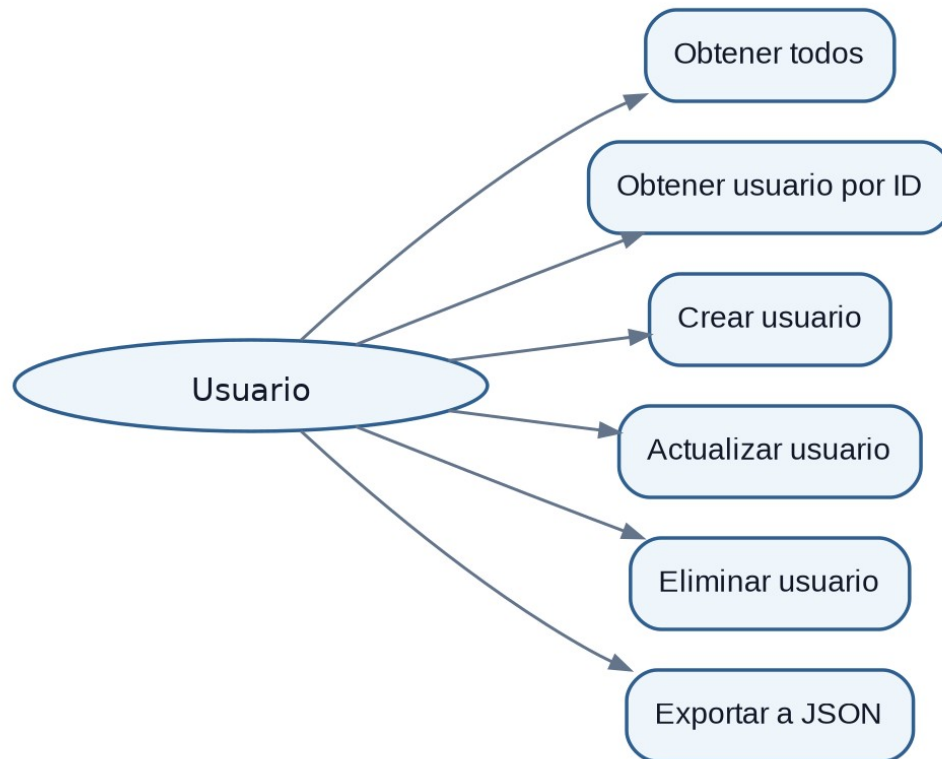
docker compose --profile prod up -d --build
```

En el perfil prod se arrancan tres servicios: backend, PostgreSQL y Redis. PostgreSQL tiene healthcheck y los datos se mantienen en volúmenes de Docker.

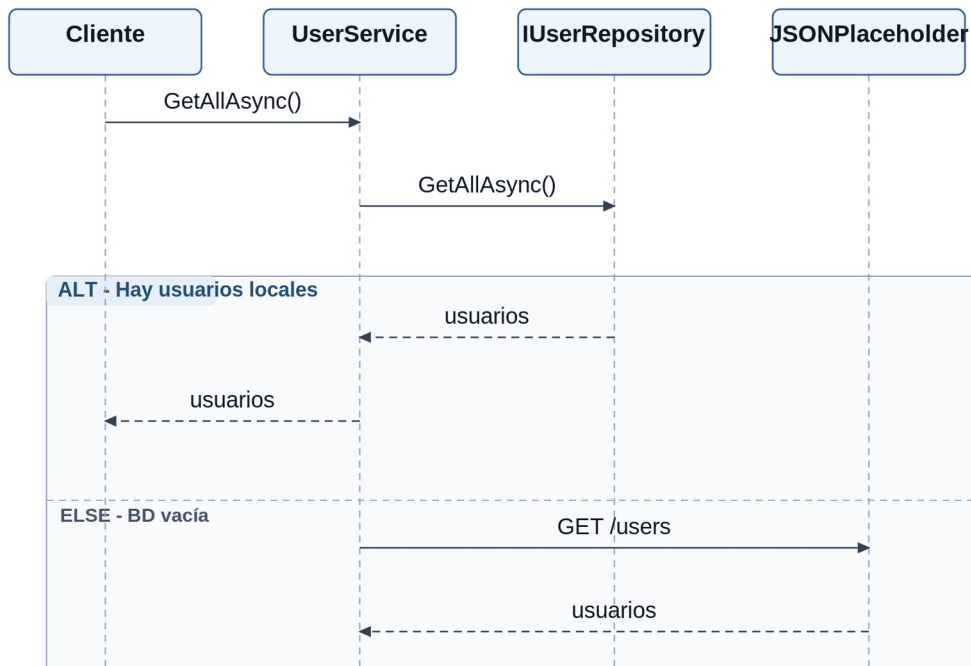
12. Diagramas

Los diagramas se han definido en Mermaid y se muestran a continuación de forma gráfica para que el flujo se pueda leer fácilmente.

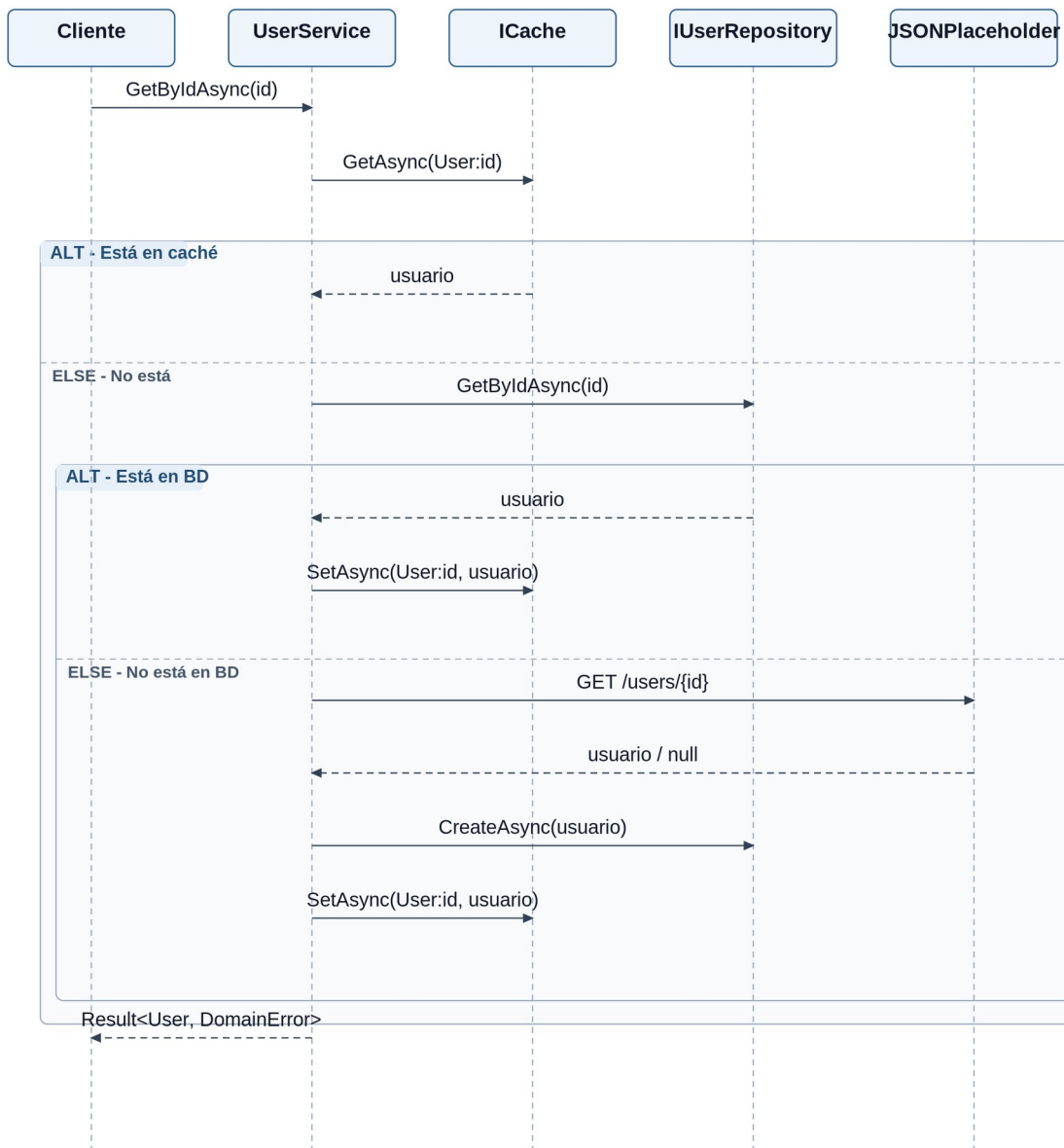
12.1 Casos de uso



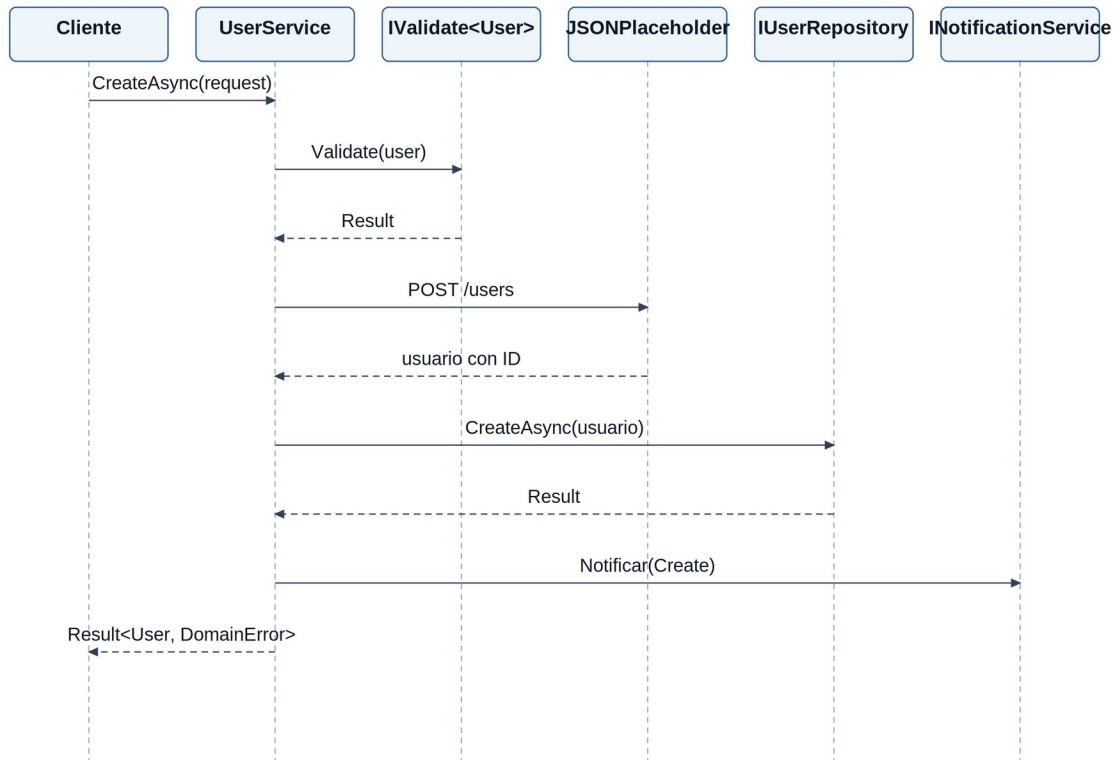
12.2 Obtener todos los usuarios



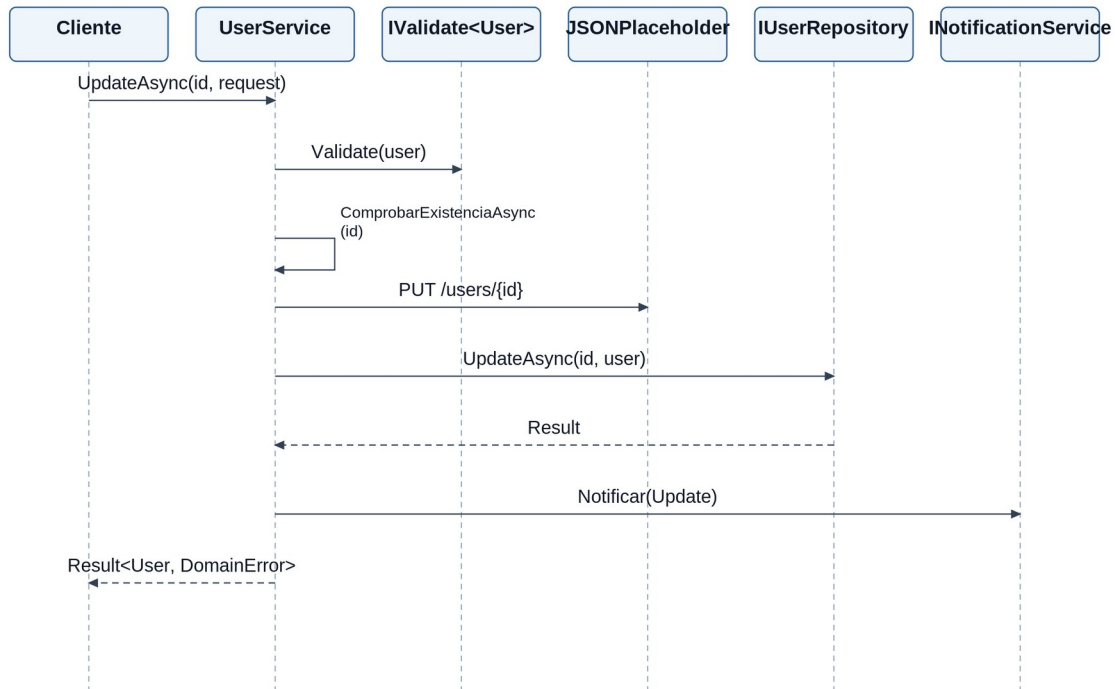
12.3 Obtener usuario por ID



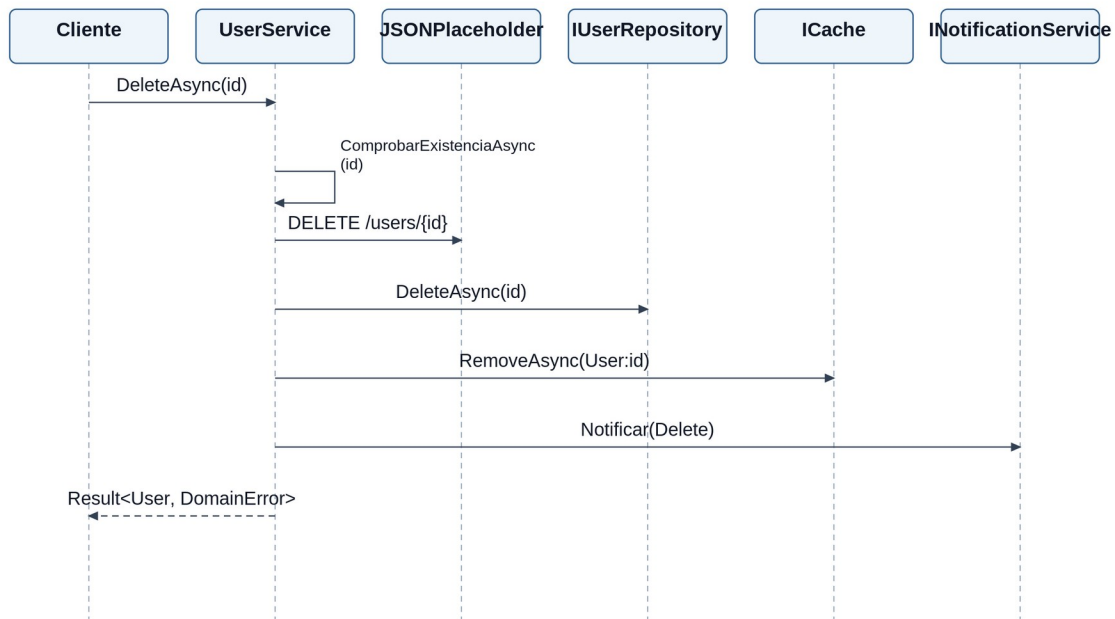
12.4 Crear usuario



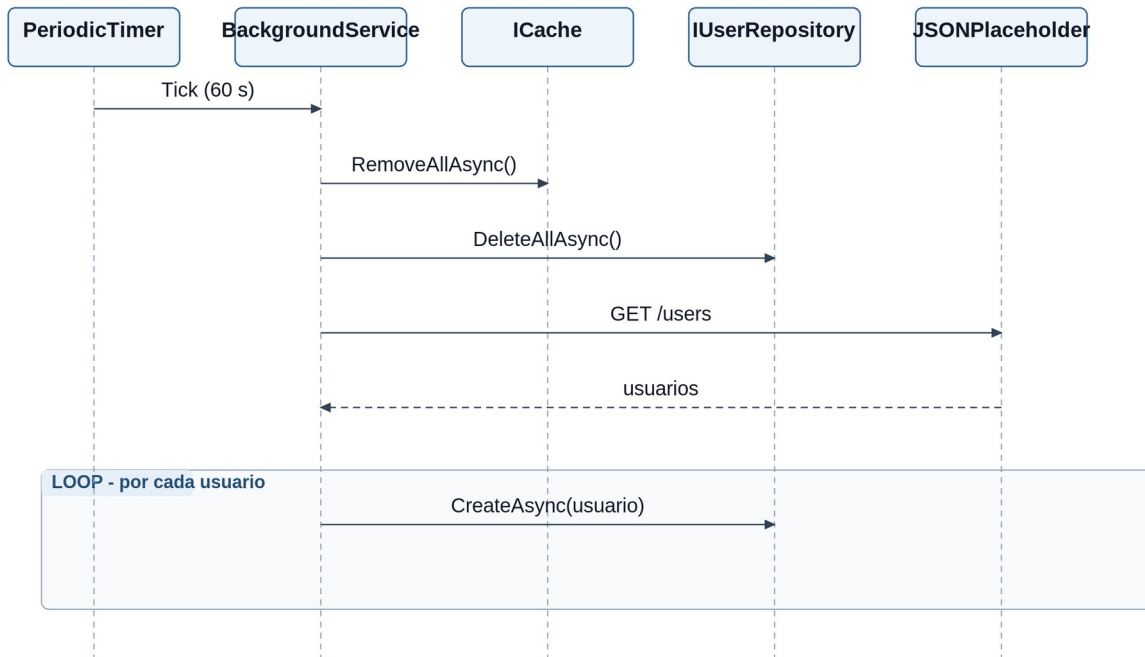
12.5 Actualizar usuario



12.6 Eliminar usuario



12.7 Sincronización cada 60 segundos



13. Conclusiones

La práctica nos ha permitido juntar varias partes vistas en clase dentro de un mismo proyecto: consumo de una API con Refit, persistencia con EF Core, caché, inyección de dependencias, programación reactiva, tareas periódicas, Docker y testing.

La parte más útil del diseño es poder cambiar la infraestructura sin modificar la lógica principal. En Development se puede trabajar de forma sencilla con SQLite y MemoryCache, mientras que en Production se utilizan PostgreSQL y Redis. También hemos separado la validación, los errores y la exportación a JSON para que cada clase tenga una responsabilidad concreta.

Finalmente, los tests y el informe de cobertura nos sirven para comprobar no solo que la aplicación funciona, sino también qué partes del código quedan realmente verificadas.